上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/10/9

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

软件的生命周期

模块 5 · 重新认识软件（1/4）

饮 水 思 源 • 爱 国 荣 校

SJTU

重新认识软件 – 软件的生命周期

本讲位置：从「结构」到「时间轴」

上一讲 · L-04

架构案例分析：电商系统从单机到云原生的 11 个阶段

把零件装成系统：每一步都是瓶颈 → 方案 → 新问题

看的是软件的「结构」：由什么组成、怎么连

本讲问题

一行代码从写下到退役，经过哪些状态？

每个状态的产出物是什么？能不能回到上一个状态？

钩子

选课平台今天 9:00 开放选课。8:55，刚上线的新功能有 bug——你能做什么？

课末回答

本讲结构

1 为什么要重新认识软件

2 制品的生命周期

3 实例的生命周期

4 谁负责

5 课程实践：选课平台的一次发布

本模块 重新认识软件（L-05 ~ L-08）：一个有生命周期的制品（本讲）· 一个运行中的进程（L-06）· 一个资源的消费者（L-07）· 一组可度量的数（L-08）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、为什么要重新认识软件

第五讲 · 软件的生命周期

饮 水 思 源 • 爱 国 荣 校

SJTU

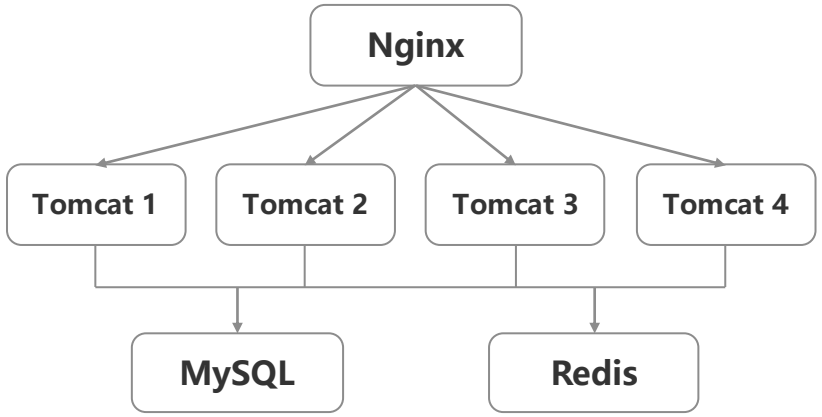
为什么重新认识

框图里的软件 vs 时间轴上的软件

框图里的软件 = 结构

问的是：由什么组成、怎么连

开发者默认看这张图



一张图回答「有哪些方块、怎么连」

时间轴上的软件 = 一连串状态

问的是：现在处于什么状态、怎么到下一个状态、能不能回去

运维默认看这张图



一天若干次；每个版本都会被下一个替换

要点 DevOps 要求两张图都看；L-01 讲的 SaaS、持续、长生命周期、迭代，全在右图上

来源：L-01、L-04 讲义

重新认识软件 – 软件的生命周期

为什么重新认识

两条时间轴：制品的生命周期，实例的生命周期

制品的生命周期 · 版本之间 · 跨天到跨年

源码 → 构建 → 测试 → 版本 → 部署 → 运行 → 变更 → 下线

实例的生命周期 · 一次运行之内 · 跨分钟到跨周（在「运行」段里重复很多次）

启动 → 就绪 → 服务 → 停止 → 替换

...替换之后，新实例从「启动」再来一遍

制品轴回答

这个版本是什么、在哪些环境、能不能退回去

实例轴回答

这个进程现在能不能接流量、怎么安全地停

要点 两条轴的交点就是「部署」：把某个制品变成若干实例。后面每一页都归到其中一条轴上

数字直觉：DORA 四个指标

指标	衡量什么	精英团队
部署频率	多久部署一次到生产	按需，一天多次
变更前置时间	从提交到生产要多久	小于一天
变更失败率	部署后需要修复 / 回滚的比例	约 5%
失败部署恢复时间	坏了多久能恢复	小于一小时

怎么读这四个数

前两个是「速度」，后两个是「稳定」；DORA 的发现是两者不矛盾——做得好的团队两边都好

要点 这四个数就是生命周期本身的健康指标——L-08 讲指标时回收

来源：DORA, Accelerate State of DevOps Report（dora.dev）；数值以最新年度报告为准

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、制品的生命周期

第五讲 · 软件的生命周期

饮 水 思 源 • 爱 国 荣 校

SJTU

制品 · 总览

制品生命周期总览：九个节点，每个都有产出物



产出物是什么?

没有产出物的阶段不算阶段；产出物决定了它能不能被追溯、被复现

常见问题是什么?

每个节点按「问题 → 原因 → 对策」看

能不能回到上一个状态?

回不去的变更要格外小心，放最后、等最久

读法 每个箭头都是一次状态变更；本节逐个节点问这三个问题

源码与提交

产出物：一次提交

生命周期的最小变更单位

之后的一切——制品、版本、部署——都要能追溯到它

两道门禁

评审：人看

CI：机器看——构建与测试通过才能合入主干

L-02 CI 原则「每日提交」「自检构建」的落点

原则

分支短命、小步提交

主干随时可发布

分支模型（主干开发 vs 长分支、分支保护）的做法见实验 1 指导书

要点 提交是追溯的起点：「线上跑的是哪次提交」从这里开始有答案

来源：L-02 Part A（CI 原则）

构建与制品：不可变制品

问题	原因	对策
「在我机器上能跑」	构建与运行环境不一致：依赖版本、系统库、JDK 小版本、环境变量	构建一次、处处部署（build once, deploy many）；制品不可变，环境差异只允许来自配置
同一份源码两次构建结果不同	依赖没锁定（「最新版」）；基础镜像用 latest	锁文件（package-lock / go.sum / Maven 固定版本）；镜像用 digest 引用；构建环境本身也是镜像
制品库塞满，找不到线上跑的是哪个	没有保留策略；制品没和提交对应	制品带 git SHA 与构建号；按规则保留：最近 N 个 + 所有已上线的

要点 制品是生命周期的「不动点」：从这里起不再改变，只被加配置、被放进环境；容器镜像把程序 + 依赖 + 运行环境一起冻住（回指 L-02）

来源：The Twelve-Factor App, II. Dependencies；V. Build, release, run

测试：放在生命周期哪一段

构建时	test 环境	staging	线上
单元测试 · 秒级 · 最多	集成测试 · 分钟级	端到端、性能 · 小时级	合成监控、金丝雀 · 最少

问题	原因	对策
发布前集中测两天，发现问题回去改、再测两天	反馈环太长	测试前移：越便宜的测试越早跑；门禁分层——构建门禁、合并门禁、发布门禁
测试环境测过了，上线还是坏	环境漂移	同一制品在各环境跑 + 环境用代码定义（后面两页）

要点 测试不是一个阶段，是贯穿每一次状态变更的门禁

来源：L-02 Part A（测试自动化）；M. Fowler, TestPyramid

版本：给制品起名字

MAJOR	MINOR	PATCH
不兼容的改动	向后兼容的新功能	向后兼容的修复

SemVer：主版本.次版本.修订号，如 1.4.2

标识	谁用	回答什么
版本号 1.4.2	依赖方、用户	兼容性承诺
构建号 #1873	流水线	哪一次构建
git SHA a1b2c3d	开发者	哪一份源码

三者要能互查：制品同时打三个标签；「线上跑的是什么」必须一句话能答

要点 SemVer 的意义不在数字，在承诺：MAJOR 变了就是告诉依赖方「你要改代码」——Tomcat 10 的 javax → jakarta 就是一次 MAJOR（L-04）

来源：semver.org；Apache Tomcat Migration Guide

三个动词：发布 / 部署 / 放量

动词	改变哪一层的状态	例子	撤销方式
发布 release	制品层：这个制品获得版本号，可以被部署	打 tag v1.4.2，推到制品库	撤回版本（很少做）
部署 deploy	环境层：制品被放进某个环境并运行	v1.4.2 跑在 prod 的 4 台机器上	把上一版部署回去
放量 rollout	流量层：多少用户用上了新版本	1% 用户走新版本；功能开关打开	切回流量 / 关开关

L-02 的 Continuous Delivery（随时可发布）vs Continuous Deployment（自动部署）就是前两层的区别

把「部署」和「放量」拆开，是互联网产品最重要的生命周期技巧：代码可以先进生产而不对用户可见（dark launch）

要点 撤销速度自上而下递增：切回流量以秒计，重新部署以分钟计——课程实践会用到

注：英文 release 既指「打版本」也指「对用户开放」，本课按上表区分。来源：Humble & Farley (2010)；Hodgson, Feature Toggles (2017)

环境与配置：同一制品，不同配置

dev	test	staging	prod
开发者自己的环境	跑自动化测试	尽量与生产一致，做预演	生产

问题	原因	对策
数据库地址、密钥写在代码里，换环境要改代码重新构建	配置没和代码分离	配置从环境注入（环境变量 / 配置中心）；密钥单独管理，不进仓库
改了一个配置项，服务全挂	配置变更没当作变更对待：没评审、没灰度、没回滚	配置也走生命周期：版本化、评审、先灰度后全量、可回滚（Q-2 第五幕的「配置失误」）
staging 测过没问题，prod 不一样	环境是手工搭的，日久漂移	环境用代码定义（IaC），重建环境而不是修环境——模块 6/7 展开

要点 生命周期里真正可变的只有两样：制品的版本、配置的版本——都要能答「现在是哪一版、上一版是什么」

来源：The Twelve-Factor App, III. Config；X. Dev/prod parity

部署策略对比：解决了什么，增加了什么成本

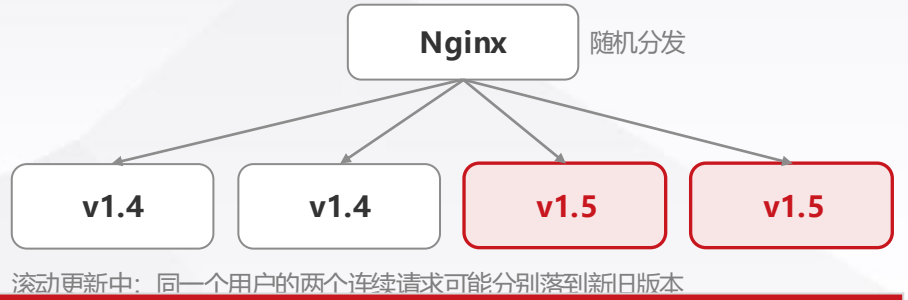
策略	做法	解决了什么	增加了什么成本	回退速度
重建（停机）	停旧、起新	最简单	停机窗口，用户感知	分钟：再部署一次
滚动	一次换一台（或一批）	不停机	新旧版本共存 → 必须向后兼容；期间容量下降	分钟：反向滚动
蓝绿	两套环境，切流量	切换与回退都是秒级	资源翻倍（至少在切换窗口内）；数据库只有一套，schema 仍要兼容	秒：切回
金丝雀	先给 1%～10% 流量看指标，再放量	用真实流量验证，影响面可控	需要按比例切流量的设施和对比指标的能力；新旧共存更久	秒：切回流量
功能开关	代码已部署，功能由配置决定是否生效	部署与放量彻底分开；回退最快	代码里多了分支；开关要清理，否则成技术债；开关本身是配置变更，要管	秒：关开关

要点 没有最好的策略，看回退速度要求和愿付的成本；实际常组合：滚动 + 金丝雀 + 开关（回指 L-02 部署模式、L-03 灰度与回滚）

来源：M. Fowler, BlueGreenDeployment (2010)；D. Sato, CanaryRelease (2014)

制品 • 运行

版本共存与向后兼容



问题	原因	对策
第一个请求落在 v1.5 拿到新格式数据，第二个落在 v1.4 解析失败	API / 数据格式不向后兼容	只加字段不删不改语义；删字段先标废弃、过几个版本再删；消费者容忍未知字段（tolerant reader）
v1.5 写进队列的消息，v1.4 的消费者读不懂	消息格式变了	消息带版本号；消费者先升级、生产者后升级（先能读新的，再写新的）
v1.5 改了 session / 缓存格式，v1.4 读到崩溃	共享状态的格式变了	换 key 前缀，或兼容读

要点 只要不停机，新旧版本共存就是常态；「向后兼容」不是美德，是生存条件——它的另一半 schema 在后面

来源：M. Fowler, TolerantReader (2011)

回滚的边界

能回滚的——代码与配置层

制品：重新部署上一版

配置：切回上一版配置

流量：切回旧版本 / 关开关

不能自动回滚的——状态

schema：删掉的列回不来；NOT NULL 让旧代码写不进去

已写入的数据：新逻辑写的记录，旧代码不认识

已发出的消息：短信发了就是发了

缓存：新格式的项还在，旧代码读到就错

外部副作用：调了第三方接口、扣了钱

回滚的真正含义

让系统回到一个已知正确的状态；状态 = 代码 + 配置 + 数据

两种修复

回滚（roll back）vs 向前修复（fix forward）；改了数据的变更往往只能向前修

为回滚而设计

改代码和改数据拆成不同的发布；写数据前先问「旧代码读到这条会怎样」

要点 代码和配置可以版本化回退，数据只能向前走

来源：Google SRE Book, Ch. 8 Release Engineering；Humble & Farley, Continuous Delivery, Ch. 12 Managing Data

schema 变更：expand–contract

- 场景：enrollment 表要从「按课程」改为「按教学班」判冲突，加 section_id 列

① 扩展 expand

加可空列 section_id

只改 schema，不改代码

旧代码不受影响

回滚：不需要——schema 向后兼容

→

② 双写 migrate

新代码写两列，后台回填历史数据

读仍走旧列

回滚：换回旧代码，新列留着不碍事

→

③ 切读

新代码改为读 section_id

此时数据已齐

回滚：换回上一版——读旧列，旧列仍在双写

→

④ 收缩 contract

确认没人再读旧列

再删旧列 / 加 NOT NULL

不可回滚：所以放最后、等最久

时间向右：每步一个发布，每步都可单独回滚；只有最后一步回不去

要点 每步都兼容前后两个版本，所以任何一步都能退回上一步；代价是周期长、双写有一致性成本。一次发布同时「改 schema + 改代码」，是回不去的发布

来源：D. Sato, ParallelChange (martinfowler.com, 2014)；Humble & Farley, Continuous Delivery, Ch. 12

下线与退役

问题	原因	对策
依赖的运行时 / 库到了 EOL，不再有安全补丁	依赖也有生命周期，而且不由你控制	盘点依赖的 EOL（endoflife.date），升级纳入路线图。例：Tomcat 8.5 移除 BIO、10.1 移除 APR（L-04）；CentOS 7 于 2024-06-30 EOL
一个老接口没人敢删，不知道还有谁在用	功能没有废弃流程	标记 deprecated → 监控调用量 → 通知调用方 → 设下线日期 → 下线 → 删代码
下线的功能留下的数据、队列、定时任务还在跑	只删了代码没删资源	下线清单：代码、配置、数据（归档或删除，注意合规保存期）、基础设施、监控告警、文档

要点 没有退役阶段的生命周期是只进不出的债务；「长生命周期」的另一面是：它会见证所有依赖的生命周期

来源：endoflife.date；Apache Tomcat Migration Guide

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

三、实例的生命周期

第五讲 · 软件的生命周期

饮 水 思 源 • 爱 国 荣 校

SJTU

重新认识软件 – 软件的生命周期

实例

一次运行的状态机



启动中

读配置、建数据库 / Redis 连接池、加载缓存或模型、JIT 预热

进程活着，但不能服务

已就绪

依赖都通了，可以接流量

服务中

正常

中途可能「活着但不健康」：线程池满、依赖断

停止中

收到停止信号 → 不再接新请求 → 处理完在途请求 → 关闭连接 → 退出

超时则被强杀

要点 制品轴上的一次「部署」，在实例轴上就是「起一批新的、停一批旧的」——每种部署策略都在编排这个状态机

来源：The Twelve-Factor App, IX. Disposability

就绪 vs 存活：健康检查的两个问题

存活：还活着吗?
进程在、能响应——不行就重启

就绪：能接流量吗?
依赖通了、预热完了、没过载——不行就先别给流量，但别重启

问题	原因	对策
滚动更新时新实例一起来就被切进流量，前几秒全是 500	只看「进程起来了」，没看「能服务了」	就绪检查通过后再加入负载均衡（Nginx / LB 健康检查；L-04 阶段③、2025 实验 3 的 /healthz）
实例僵死（死锁、OOM 后半残）但进程还在，流量照打、一直超时	没有存活检查，或只检查了进程存在	存活检查探真实路径；失败 N 次重启
数据库抖一下，所有实例的存活检查同时失败，全体重启，雪崩	把「依赖不可用」当成「自己死了」	依赖故障只影响就绪，不影响存活；存活检查不依赖外部

要点 两个问题的答案不同，处理也不同；Kubernetes 把它们做成 livenessProbe / readinessProbe（另有 startupProbe）——模块 7 对上

来源：Kubernetes docs, Liveness, Readiness and Startup Probes

优雅停止，与启动时间

优雅停止的五步，与每步的坑

1 收到 SIGTERM（不是 SIGKILL）——进程要监听它

2 从负载均衡摘除 / 就绪置为失败——否则还会来新请求

3 等在途请求完成——要有上限（如 30 s），否则一个慢请求拖住整个发布

4 关闭连接池、刷日志、退出

5 超时仍不退出 → 强杀，在途请求丢失

代价

发布时间 = 实例数 ×（停止 + 启动 + 就绪）/ 并行度

长连接（WebSocket、SSE，L-03）要主动通知客户端重连，drain 更慢

启动时间为什么是硬参数

决定扩容反应速度：流量来了，新实例几分钟后才能用

决定回滚速度；决定能不能用 Serverless（冷启动）

L-03 的 Quarkus / GraalVM 对比就是为缩短它——下一讲展开

要点 停得优雅、起得快，是实例能被频繁替换的前提

来源：The Twelve-Factor App, IX；Kubernetes docs, Pod Lifecycle（默认终止宽限期 30 s）

实例短命倒逼无状态

实例随时会被替换
部署、扩缩容、故障、机器回收

→

放在实例里的东西随时会丢
内存里的 session、本地文件、本地缓存

→

状态必须放到实例之外
Redis / 数据库 / 对象存储

→

实例才可以随便杀、随便起

→

才可能滚动、扩缩容、自动恢复

回指

L-04 阶段③加 Nginx 后的「Session 共享」问题，根源不是负载均衡，而是实例不再唯一、不再长寿

L-02 云原生：无状态、水平扩展

推论

无状态不是架构偏好，是实例生命周期缩短后的必然结果

反过来，有状态的东西（数据库、队列）实例生命周期就得长、替换就得谨慎——模块 6 要区分的两类工作负载

要点 无状态是「实例短命」倒逼出来的设计，不是风格选择

来源：The Twelve-Factor App, VI. Processes

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

四、谁负责

第五讲 · 软件的生命周期

饮 水 思 源 • 爱 国 荣 校

SJTU

谁负责

各阶段的所有者 • 流水线 = 生命周期的自动化

阶段	传统所有者	DevOps 下的所有者
源码 / 构建 / 测试	开发	开发
版本 / 发布	发布工程 / 测试	开发（自动化）
环境 / 配置 / 部署	运维	开发 + 平台（自助）
运行 / 监控 / 值班	运维	开发（you build it, you run it）+ SRE
下线	没人	产品 + 开发

流水线 = 这条生命周期的自动化

每一个箭头是一个流水线阶段，每一个门禁是一次自动检查；L-02 讲怎么搭，本讲讲它在管什么

要点 DevOps 不是一个岗位，是把生命周期的所有权交回写代码的人，平台团队提供自助工具（回指 L-02「DevOps 与架构」）

来源：L-02 讲义；Google SRE Book, Ch. 8

实验 1 预告：在 GitHub 上搭一条从提交到制品的流水线

做什么

代码仓 + 分支保护（合并门禁）

分支模型的做法在指导书里讲

GitHub Actions：push → 构建 → 测试 → 打包制品 / 容器镜像 → 打版本 / release

产出：一个能回答「线上跑的是哪次提交」的制品

对应本讲

源码与提交 → 构建与制品 → 测试 → 版本 → 发布

每一步都是一次状态变更，流水线把它们自动化

第二节前五页

注 时间与细节以指导书为准

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

五、课程实践：选课平台的一次发布

第五讲 · 软件的生命周期

SJTU

场景设定：开放日前的一次发布

选课平台（思考题 1 时期）

Nginx + 4 台 Tomcat + 1 台 MySQL（+ Redis 存 session）

10/9 9:00 开放选课，峰值 ≈ 秒杀（L-04）

要上线的变更：冲突检测新规则——按教学班判时间冲突，需要 enrollment 表加 section_id 列，并修改选课接口

时间线：10/6 代码完成 → ？ 发布 → 10/9 9:00 开放

已知参数

一台 Tomcat 启动 + 预热约 1 min；优雅停止上限 30 s

蓝绿需要再开 4 台（按天计费）

功能开关：有配置中心可热生效；只有配置文件则需重启生效——两种都讨论

钩子 8:55，发现新规则有 bug。你能做什么？

课堂讨论

1 变更冻结

开放日前几天冻结？冻结什么（代码 / 配置 / 基础设施）？

冻结的代价是什么、例外流程怎么定？

2 schema 变更

这次加列怎么做才能回滚？

拆几步、各在哪天发？

3 8:55 发现新规则把「单双周」课程误判为冲突

回滚（4 台滚动要几分钟？蓝绿呢？）还是关功能开关？

哪个更快、各自的前提是什么？

4 回滚之后，8:50 ~ 8:55 之间的请求怎么办

(a) 误拒了合法选课

(b) 反过来，误收了冲突选课

讲法 四题各 2 分钟小组讨论、1 分钟汇报，剩余时间讲评

讨论要点

1 冻结

代码冻结 2～3 天，给 staging 预演和观察留时间；配置变更允许但需双人评审；基础设施冻结

例外 = hotfix 流程：最小改动 + 两人批准 + 回滚预案

冻结不是「不许改」，是「改动要付更高的门票」

2 expand–contract

10/6 发布①加可空列（纯 schema）；10/7 发布②双写 + 回填 + 新规则藏在开关后（默认关）

10/8 staging 预演开关打开；10/9 之后再切读与收缩

任何一步出问题都退回上一步，新列留着不碍事

3 回退速度

开关（热生效）≈ 秒级，前提是 10/7 已把开关做进去、8:55 有人有权限且知道去关；要重启生效就退化成滚动

蓝绿 ≈ 秒级，前提是绿环境没撤（多付一天 4 台）

滚动回滚 ≈ 4 ×（0.5 + 1）min ≈ 6 min；两台一批 ≈ 3 min，但峰值前容量减半——9:00 前刚好赶上，风险高

4 副作用

(a) 误拒：没有写入，学生重试即可，但要公告

(b) 误收：冲突记录已在库里，回滚代码不会删它们——找出 8:50～8:55 按新规则写入的记录（写入要带版本 / 规则标记）→ 逐条补偿（取消 + 通知，回指 L-03 Saga）→ 名额归还

结论 回退速度是发布前买的，不是出事时想出来的；代码能回滚，副作用只能补偿——8:55 能做什么，取决于 10/6～10/8 做了什么

本讲小结

- **软件是时间轴上的东西**
  - 制品轴（版本之间）与实例轴（一次运行之内），部署是两轴的交点
- **生命周期里可变的只有两样**
  - 制品的版本、配置的版本——制品不可变、配置分离、环境一致
- **回滚有边界**
  - 发布、部署、放量三层撤销速度递增；代码和配置能回，数据只能向前
- **为回滚而设计**
  - schema 用 expand–contract，功能用开关，版本共存靠向后兼容
- **实例短命 → 无状态 → 才能滚动、扩缩容、自愈**

判断标准 这个阶段的产出物是什么？能不能回到上一个状态？（与「解决了什么问题、增加了什么成本」并列）

课后思考与参考资料

课后思考

1. 画出你小组项目的制品生命周期图（九个节点），标出哪几段已自动化、哪几段还是手工；「线上跑的是哪次提交」你们现在能不能一句话回答?

2. 找一个你用过的软件 / 库 / 运行时的 EOL 公告，说明它对使用者的生命周期提出了什么要求、迁移成本在哪。

3. 选课平台里，哪些变更可以用功能开关、哪些不能?（提示：开关能切流量层，切不了 schema 和已写数据。）

参考资料

The Twelve-Factor App（12factor.net）：III Config、V Build-release-run、VI Processes、IX Disposability、X Dev/prod parity

D. Sato, ParallelChange, martinfowler.com (2014)；M. Fowler, BlueGreenDeployment (2010)、TolerantReader (2011)

P. Hodgson, Feature Toggles, martinfowler.com (2017)

J. Humble & D. Farley, Continuous Delivery (2010), Ch. 12

DORA, Accelerate State of DevOps Report（dora.dev）

Google SRE Book, Ch. 8 Release Engineering

Kubernetes docs: Liveness, Readiness and Startup Probes；Pod Lifecycle

endoflife.date

感谢聆听

饮 水 思 源　爱 国 荣 校